

EC-350 AI and Decision Support Systems

Week 14 Multilayer Neural Networks/ Multilayer Perceptron (MLP)

Dr. Arslan Shaukat



Acknowledgements: Lecture slides material from
Duda, Hart and Stork, Dr. Gavin Brown

1

Introduction

- Goal: Classify objects by learning nonlinearity
 - *There are many problems for which linear discriminants are insufficient for minimum error*
 - *There is no automatic method for determining the nonlinearities when no information is provided to the classifier*
 - *In using the Multilayer Neural Networks, the form of the nonlinearity is learned from the training data*

2

Feedforward Operation and Classification

- A three-layer neural network consists of an input layer, a hidden layer and an output layer interconnected by modifiable weights represented by links between layers
- A single “bias unit” is connected to each unit other than the input units
- Net activation:

$$net_j = \sum_{i=1}^d x_i w_{ji} + w_{j0} = \sum_{i=0}^d x_i w_{ji} \equiv w_j^t \cdot X,$$

where the subscript i indexes units in the input layer, j in the hidden; w_{ji} denotes the input-to-hidden layer weights at the hidden unit j . (In neurobiology, such weights or connections are called “synapses”)

- Each hidden unit emits an output that is a nonlinear function of its activation, that is: $y_j = f(net_j)$

06/01/2023

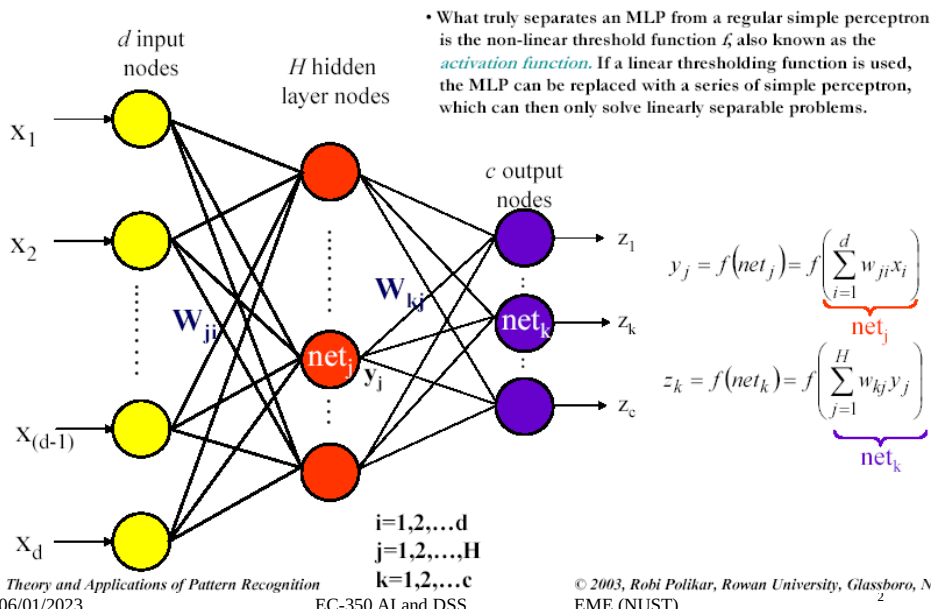
EC-350 AI and DSS

EME (NUST)

2

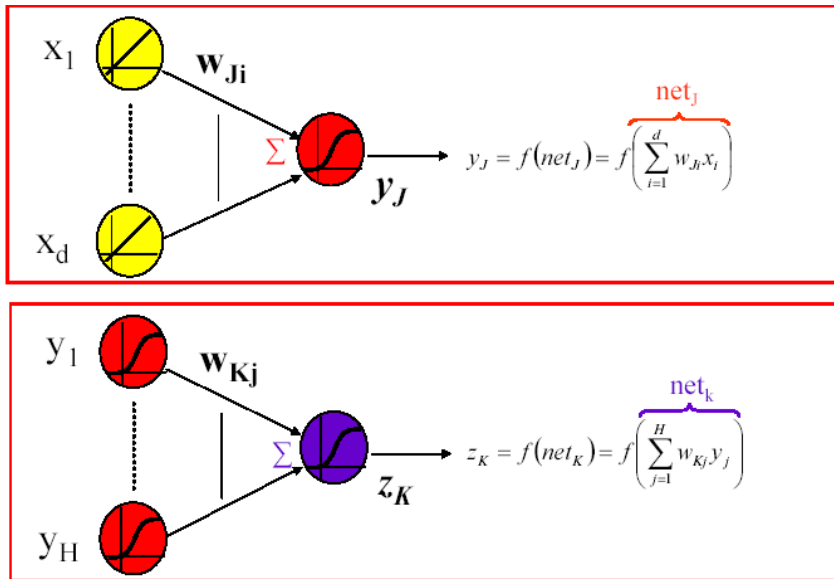
3

MLP: Three Layered Network



4

MLP: Three Layered Network



06/01/2023

EC-350 AI and DSS

EME (NUST)

5

MLP: Three Layered Network

- The function $f(\cdot)$ is also called the activation function or “nonlinearity” of a unit. There are more general activation functions with desirable properties
- Each output unit similarly computes its net activation based on the hidden unit signals as:

$$net_k = \sum_{j=1}^{n_H} y_j w_{kj} + w_{k0} = \sum_{j=0}^{n_H} y_j w_{kj} = w_k^t \cdot y,$$

where the subscript k indexes units in the output layer and n_H denotes the number of hidden units

- More than one output are referred z_k . An output unit computes the nonlinear function of its net, emitting $z_k = f(net_k)$
- In the case of c outputs (classes), we can view the network as computing c discriminant functions $z_k = g_k(x)$ and classify the input x according to the largest discriminant function $g_k(x) \forall k = 1, \dots, c$.

06/01/2023

EC-350 AI and DSS

EME (NUST)

2

6

Practical Considerations: Activation Function

- Desirable properties of an activation function
 - Nonlinearity—gives the multilayer networks the power of generating nonlinear decision boundaries;
 - Saturation for classification problems—so that the outputs can be limited between some minimum and maximum limits (-1 and 1, or 0 and 1).
 - Continuity and smoothness—so that we can take its derivative
 - Monotonic—so that the activation function itself does not introduce additional local minima
 - Linearity for small values of net, to preserve the properties of linear discriminant functions
- A function that satisfies all of the above: Sigmoid

06/01/2023

EC-350 AI and DSS

EME (NUST)

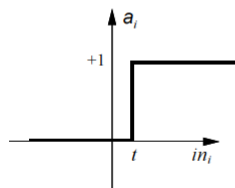
7

Activation Functions

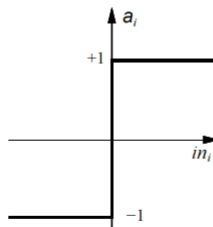
The **activation function** g calculates the output a_i (from the inputs) which will be transferred to other units via output-links:

$$a_i := g(in_i)$$

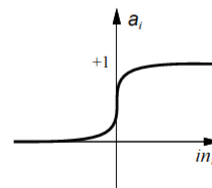
Examples:



(a) Step function



(b) Sign function



(c) Sigmoid function

06/01/2023

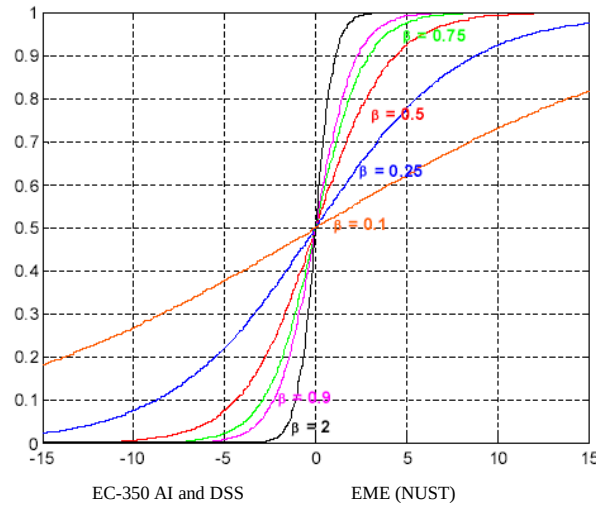
EC-350 AI and DSS

EME (NUST)

8

Sigmoid Activation Function

Logarithmic Sigmoid
$$f(\text{net}) = \frac{1}{1 + e^{-\beta \cdot \text{net}}}$$



06/01/2023

EC-350 AI and DSS

EME (NUST)

9

XOR: Implementation by 3-layered Network

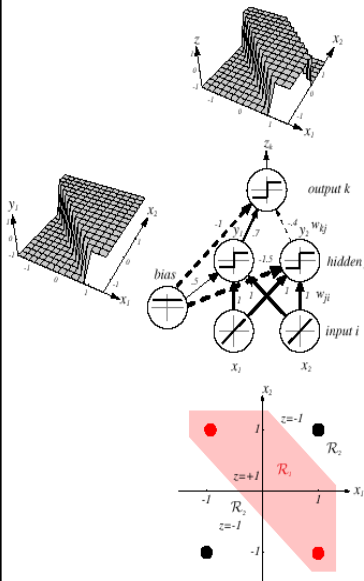


FIGURE 6.1. The two-bit parity or exclusive-OR problem can be solved by a three-layer network. At the bottom is the two-dimensional feature x_1, x_2 -space, along with the four patterns to be classified. The three-layer network is shown in the middle. The input units are linear and merely distribute their feature values through multiplicative weights to the hidden units. The hidden and output units here are linear threshold units, each of which forms the linear sum of its inputs times their associated weight to yield net , and emits a +1 if this net is greater than or equal to 0, and -1 otherwise, as shown by the graphs. Positive or "excitatory" weights are denoted by solid lines, negative or "inhibitory" weights by dashed lines; each weight magnitude is indicated by the line's thickness, and is labeled. The single output unit sums the weighted signals from the hidden units and bias to form its net , and emits a +1 if its net is greater than or equal to 0 and emits a -1 otherwise. Within each unit we show a graph of its input-output or activation function— $f(\text{net})$ versus net . This function is linear for the input units, a constant for the bias, and a step or sign function elsewhere. We say that this network has a 2-2-1 fully connected topology, describing the number of units (other than the bias) in successive layers. From: Richard O. Duda, Peter E. Hart, and David G. Stork, *Pattern Classification*. Copyright © 2001 by John Wiley & Sons, Inc.

06/01/2023

EC-350 AI and DSS

EME (NUST)

1

10

XOR: Implementation by 3-layered Network

- The three-layer network with the weights solves the XOR problem

$$f(\text{net}) = \text{sgn}(\text{net}) \equiv \begin{cases} 1 & \text{if } \text{net} \geq 0 \\ -1 & \text{if } \text{net} < 0 \end{cases}$$

- The hidden unit y_1 computes the boundary:

$$x_1 + x_2 + 0.5 = 0 \begin{cases} \geq 0 \Rightarrow y_1 = +1 \\ < 0 \Rightarrow y_1 = -1 \end{cases}$$

- The hidden unit y_2 computes the boundary:

$$x_1 + x_2 - 1.5 = 0 \begin{cases} \geq 0 \Rightarrow y_2 = +1 \\ < 0 \Rightarrow y_2 = -1 \end{cases}$$

- The final output unit emits z_1

$$0.7y_1 - 0.4y_2 - 1 = 0 \begin{cases} \geq 0 \Rightarrow z_1 = +1 \\ < 0 \Rightarrow z_1 = -1 \end{cases}$$

06/01/2023

EC-350 AI and DSS

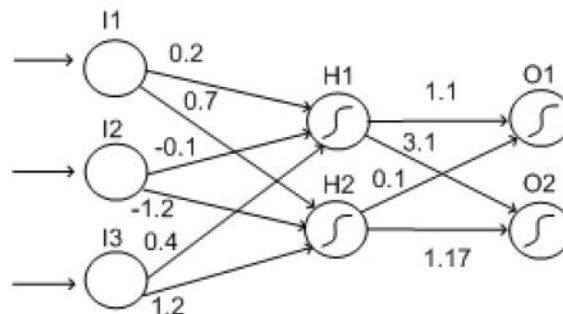
EME (NUST)

2

11

Example

- Consider the MLP:



06/01/2023

EC-350 AI and DSS

EME (NUST)

12

Example

- Suppose input values are 10, 30, 20
- The weighted sum coming into H1
 - $S_{H1} = (0.2 * 10) + (-0.1 * 30) + (0.4 * 20) = 2 - 3 + 8 = 7.$
- The σ function is applied to S_{H1} :
 - $\sigma(S_{H1}) = 1/(1+e^{-7}) = 1/(1+0.000912) = 0.999$
- Similarly, the weighted sum coming into H2:
 - $S_{H2} = (0.7 * 10) + (-1.2 * 30) + (1.2 * 20) = 7 - 36 + 24 = -5$
- σ applied to S_{H2} :
 - $\sigma(S_{H2}) = 1/(1+e^5) = 1/(1+148.4) = 0.0067$

06/01/2023

EC-350 AI and DSS

EME (NUST)

13

Example

- Now the weighted sum to output unit O1 :
 - $S_{O1} = (1.1 * 0.999) + (0.1 * 0.0067) = 1.0996$
- The weighted sum to output unit O2:
 - $S_{O2} = (3.1 * 0.999) + (1.17 * 0.0067) = 3.1047$
- The output sigmoid unit in O1:
 - $\sigma(S_{O1}) = 1/(1+e^{-1.0996}) = 1/(1+0.333) = 0.750$
- The output from the network for O2:
 - $\sigma(S_{O2}) = 1/(1+e^{-3.1047}) = 1/(1+0.045) = 0.957$
- The input (10, 30, 20) would be categorized with O2, because this has the larger output.

06/01/2023

EC-350 AI and DSS

EME (NUST)

14

Expressive Power of Multilayered Networks

- A three layered network can be used to compute the nonlinear functions which can be represented by

$$g_k(x) \equiv z_k = f \left(\sum_{j=1}^{n_H} w_{kj} f \left(\sum_{i=1}^d w_{ji} x_i + w_{j0} \right) + w_{k0} \right) \quad (1)$$

(k = 1,...,c)

- *Hidden units enable us to express more complicated nonlinear functions and thus extend the classification*
- *We can allow the activation in the output layer to be different from the activation function in the hidden layer or have different activation for each individual unit*
- *We assume for now that all activation functions be identical*

06/01/2023

EC-350 AI and DSS

EME (NUST)

2

15

Expressive Power of Multilayered Networks

- Question: Can every decision be implemented by a three-layer network described by equation (1) ?
Answer: Yes (due to A. Kolmogorov)
- “Any continuous function from input to output can be implemented in a three-layer net, given sufficient number of hidden units n_H , proper nonlinearities, and weights.”

06/01/2023

EC-350 AI and DSS

EME (NUST)

2

16

Implementation with Sigmoid Functions

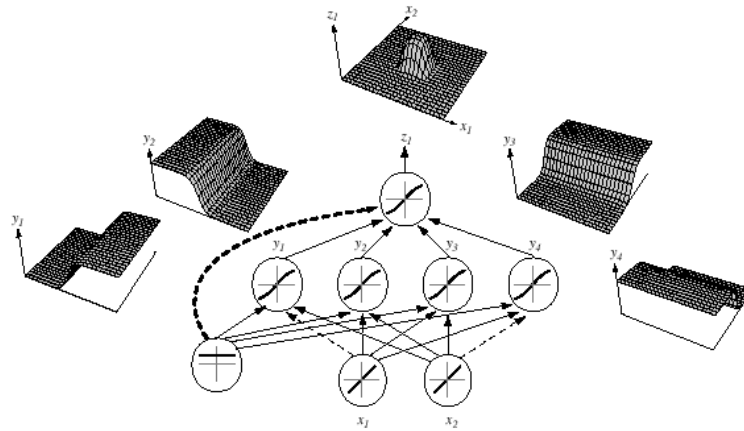


FIGURE 6.2. A 2-4-1 network (with bias) along with the response functions at different units; each hidden output unit has sigmoidal activation function $f(\cdot)$. In the case shown, the hidden unit outputs are paired in opposition thereby producing a "bump" at the output unit. Given a sufficiently large number of hidden units, any continuous function from input to output can be approximated arbitrarily well by such a network. From: Richard O. Duda, Peter E. Hart, and David G. Stork, *Pattern Classification*. Copyright © 2001 by John Wiley & Sons, Inc.

06/01/2023

EC-350 AI and DSS

EME (NUST)

2

17

Two Layered Network vs Three Layered Network

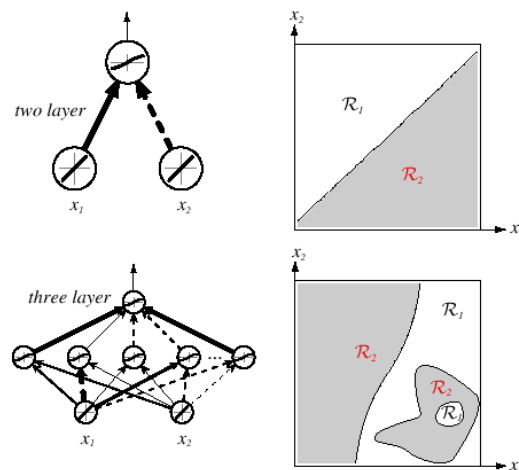


FIGURE 6.3. Whereas a two-layer network classifier can only implement a linear decision boundary, given an adequate number of hidden units, three-, four- and higher-layer networks can implement arbitrary decision boundaries. The decision regions need not be convex or simply connected. From: Richard O. Duda, Peter E. Hart, and David G. Stork, *Pattern Classification*. Copyright © 2001 by John Wiley & Sons, Inc.

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

18

Backpropagation Algorithm

- Any function from input to output can be implemented as a three-layer neural network
- This requires the nonlinear functions and the weight values which are unknown!
- Our goal now is to set the interconnection weights based on the training patterns and the desired outputs

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

19

Backpropagation Algorithm

- Network have two modes of operation:
- Feedforward
 - *The feedforward operations consists of presenting a pattern to the input units and passing (or feeding) the signals through the network in order to get outputs units (no cycles!)*
- Learning
 - *The supervised learning consists of presenting an input pattern and modifying the network parameters (weights) to reduce distances between the computed output and the desired output*

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

20

Backpropagation Algorithm

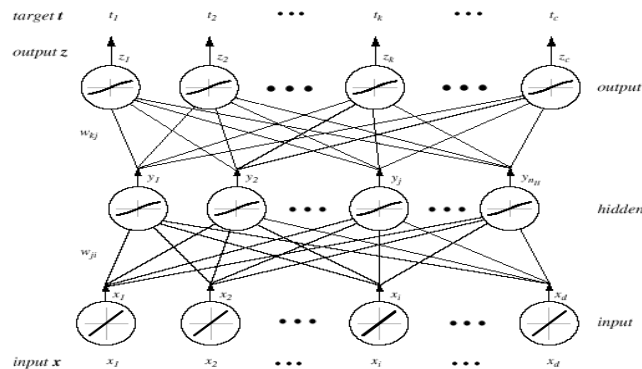


FIGURE 6.4. A $d \times n_H \times c$ fully connected three-layer network and the notation we shall use. During feedforward operation, a d -dimensional input pattern $\mathbf{x}$ is presented to the input layer; each input unit then emits its corresponding component x_i . Each of the n_H hidden units computes its net activation, net_i , as the inner product of the input layer signals with weights w_{ji} at the hidden unit. The hidden unit emits $y_j = f(net_i)$, where $f(\cdot)$ is the nonlinear activation function, shown here as a sigmoid. Each of the c output units functions in the same manner as the hidden units do, computing net_k as the inner product of the hidden unit signals and weights at the output unit. The final signals emitted by the network, $z_k = f(net_k)$, are used as discriminant functions for classification. During network training, these output signals are compared with a teaching or target vector $\mathbf{t}$, and any difference is used in training the weights throughout the network. From: Richard O. Duda, Peter E. Hart, and David G. Stork, *Pattern Classification*. Copyright © 2001 by John Wiley & Sons, Inc.

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

21

Backpropagation Algorithm: Network Learning

- Let t_k be the k -th target (or desired) output and z_k be the k th computed output with $k = 1, \dots, c$ and w represents all the weights of the network
- The training error:

$$J(w) = \frac{1}{2} \sum_{k=1}^c (t_k - z_k)^2 = \frac{1}{2} \|\mathbf{t} - \mathbf{z}\|^2$$

- The backpropagation learning rule is based on gradient descent
 - The weights are initialized with pseudo-random values and are changed in a direction that will reduce the error:

$$\Delta w = -\eta \frac{\partial J}{\partial w}$$

06/01/2023

EC-350 AI and DSS

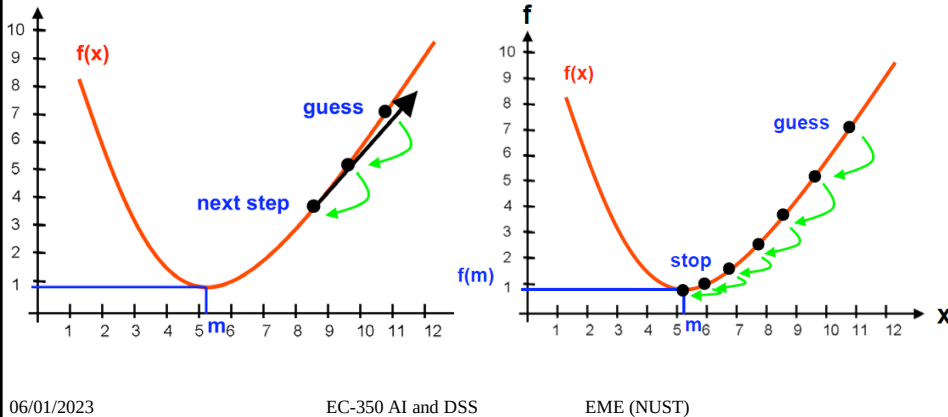
EME (NUST)

3

22

Gradient Descent

- To find a local minimum of a function using **gradient descent**, one takes steps proportional to the negative of the **gradient** of the function at the current point.



06/01/2023

EC-350 AI and DSS

EME (NUST)

23

Backpropagation Algorithm: Network Learning

- where η is the learning rate which indicates the relative size of the change in weights

$$w(m+1) = w(m) + \Delta w(m)$$

– where m is the m -th pattern presented

- Error on the hidden-to-output weights

$$\frac{\partial J}{\partial w_{kj}} = \frac{\partial J}{\partial net_k} \cdot \frac{\partial net_k}{\partial w_{kj}} = -\delta_k \frac{\partial net_k}{\partial w_{kj}}$$

– where the sensitivity of unit k is defined as: $\delta_k = -\frac{\partial J}{\partial net_k}$

– and describes how the overall error changes with the activation of the unit's net

$$\delta_k = -\frac{\partial J}{\partial net_k} = -\frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial net_k} = (t_k - z_k) f'(net_k)$$

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

24

Backpropagation Algorithm: Network Learning

Since $net_k = w_k^t \cdot y$ therefore:

$$\frac{\partial net_k}{\partial w_{kj}} = y_j$$

Conclusion: the weight update (or learning rule) for the hidden-to-output weights is:

$$\Delta w_{kj} = \eta \delta_k y_j = \eta (t_k - z_k) f'(net_k) y_j$$

■ Error on the input-to-hidden units

$$\frac{\partial J}{\partial w_{ji}} = \frac{\partial J}{\partial y_j} \cdot \frac{\partial y_j}{\partial net_j} \cdot \frac{\partial net_j}{\partial w_{ji}}$$

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

25

Backpropagation Algorithm: Network Learning

$$\begin{aligned} \text{However, } \frac{\partial J}{\partial y_j} &= \frac{\partial}{\partial y_j} \left[\frac{1}{2} \sum_{k=1}^c (t_k - z_k)^2 \right] = - \sum_{k=1}^c (t_k - z_k) \frac{\partial z_k}{\partial y_j} \\ &= - \sum_{k=1}^c (t_k - z_k) \frac{\partial z_k}{\partial net_k} \cdot \frac{\partial net_k}{\partial y_j} = - \sum_{k=1}^c (t_k - z_k) f'(net_k) w_{kj} \end{aligned}$$

Similarly as in the preceding case, we define the sensitivity for a hidden unit:

$$\delta_j \equiv f'(net_j) \sum_{k=1}^c w_{kj} \delta_k$$

which means that: “The sensitivity at a hidden unit is simply the sum of the individual sensitivities at the output units weighted by the hidden-to-output weights w_{kj} ; all multiplied by $f'(net_j)$ ”

Conclusion: The learning rule for the input-to-hidden weights is:

$$\Delta w_{ji} = \eta x_i \delta_j = \eta \underbrace{\left[\sum_{k=1}^c w_{kj} \delta_k \right] f'(net_j)}_{\delta_j} x_i$$

06/01/2023

EC-350 AI and DSS

EME (NUST)

3 3

26

Backpropagation Algorithm: Network Learning

- The weight update rule is then:

$$\Delta w_{ji} = \eta \cdot \delta_j \cdot x_i \quad \text{for hidden layer weights}$$

$$\Delta w_{kj} = \eta \cdot \delta_k \cdot y_j \quad \text{for output layer weights}$$

- In each case, the parameter δ represents the **sensitivity** of the criterion function (error) with respect to hidden / output layer node. Note from the previous expressions that the sensitivity of a hidden layer node is a weighted sum of the output sensitivities, scaled by $f'(net_j)$, where the weights are the hidden-to-output layer weights.
- The algorithm takes its name – backpropagation – from the fact that during training, the errors are propagated back, from the output to the hidden layer!

06/01/2023

EC-350 AI and DSS

EME (NUST)

27

Training Protocols

- Three major training protocols:
 - *Stochastic Learning: Instances are drawn randomly from the training data, and the weights are updated for each chosen instance.*
 - *Batch Learning: Entire training data is shown to the network before weights are updated. Each such presentation of the entire training dataset to the network is called an epoch. In this case, the error for each pattern J are computed, and summed, before weights are updated. This is the recommended mode of training an MLP.*
 - *Online Learning: Instances are drawn consecutively from the training data, and the weights are updated for each instance. This can be sensitive to the order in which the data are presented. Patterns are presented only once.*

06/01/2023

EC-350 AI and DSS

EME (NUST)

28

Backpropagation Algorithm: Stochastic Learning

- Starting with a pseudo-random weight configuration, the stochastic backpropagation algorithm can be written as:

```

Begin  initialize     $n_H$ ;  $w$ , criterion  $\theta$ ,  $\eta$ ,  $m \leftarrow 0$ 
do  $m \leftarrow m + 1$ 
    |  $x^m \leftarrow$  randomly chosen pattern
    |  $w_{ji} \leftarrow w_{ji} + \eta \delta_j x_i$ ;  $w_{kj} \leftarrow w_{kj} + \eta \delta_k y_j$ 
until  $||\nabla J(w)|| < \theta$ 
return  $w$ 

End
    
```

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

29

Back-propagation Algorithm: Batch Learning

```

Initialize  $w$ , learning rate  $\eta$ , # of hidden layer nodes  $H$ , iteration  $t=0$ , criterion  $\theta$ ;
do  $t \leftarrow t+1$  increment epoch
     $m \leftarrow 0$ ;  $\Delta w_{ji} \leftarrow 0$ ,  $\Delta w_{kj} \leftarrow 0$ ;
    do  $m \leftarrow m+1$ 
        Select pattern  $x^m$ 
         $\Delta w_{ji} \leftarrow \Delta w_{ji} + \eta \delta_j x_i$ ;  $\Delta w_{kj} \leftarrow \Delta w_{kj} + \eta \delta_k y_j$ ;
    until  $m=n$ 
     $w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$ ;  $w_{kj} \leftarrow w_{kj} + \Delta w_{kj}$ 
until  $||\nabla J(w)|| < \theta$ 
return  $w$ 

end
    
```

06/01/2023

EC-350 AI and DSS

EME (NUST)

30

Learning Curves

- Before training starts, the error on the training set is high; through the learning process, the error becomes smaller
- The average error on an independent test set is always higher than on the training set, and it can decrease as well as increase
- A validation set is used in order to decide when to stop training ; we do not want to overfit the network and decrease the power of the classifier generalization
“we stop training at a minimum of the error on the validation set”

06/01/2023

EC-350 AI and DSS

EME (NUST)

3

31

Learning Curves

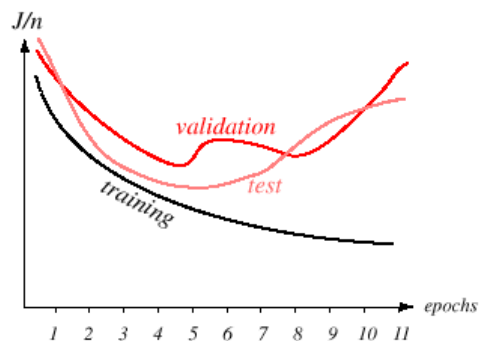


FIGURE 6.6. A learning curve shows the criterion function as a function of the amount of training, typically indicated by the number of epochs or presentations of the full training set. We plot the average error per pattern, that is, $1/n \sum_{p=1}^n J_p$. The validation error and the test or generalization error per pattern are virtually always higher than the training error. In some protocols, training is stopped at the first minimum of the validation set. From: Richard O. Duda, Peter E. Hart, and David G. Stork, *Pattern Classification*. Copyright © 2001 by John Wiley & Sons, Inc.

06/01/2023

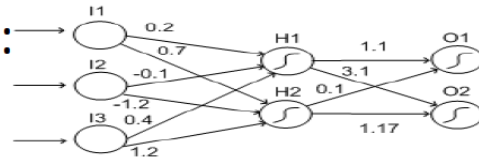
EC-350 AI and DSS

EME (NUST)

3

32

A Worked Example:



Input units		Hidden units			Output units		
Unit	Output	Unit	Weighted Sum Input	Output	Unit	Weighted Sum Input	Output
I1	10	H1	7	0.999	O1	1.0996	0.750
I2	30	H2	-5	0.0067	O2	3.1047	0.957
I3	20						

- Propagated the values (10,30,20) through the network
- Suppose now that the target categorization for the example was the one associated with O1(using a learning rate of $\eta = 0.1$)
- the target output for O1 was 1, and the target output for O2 was 0
- $t1(E) = 1$; $t2(E) = 0$; $o1(E) = 0.750$; $o2(E) = 0.957$
- error values for the output units O1 and O2
 - $\delta O1 = o1(E)(1 - o1(E))(t1(E) - o1(E)) = 0.750(1-0.750)(1-0.750) = 0.0469$
 - $\delta O2 = o2(E)(1 - o2(E))(t2(E) - o2(E)) = 0.957(1-0.957)(0-0.957) = -0.0394$

06/01/2023

EC-350 AI and DSS

EME (NUST)

33

A Worked Example:

- To propagate this information backwards to the hidden nodes H1 and H2
 - Multiply the error term for O1 by the weight from H1 to O1, then add this to the multiplication of the error term for O2 and the weight between H1 and O2, $(1.1*0.0469) + (3.1*-0.0394) = -0.0706$
- $$\delta_{H_k} = h_k(E)(1 - h_k(E)) \sum_{i \in \text{outputs}} w_{ki} \delta_{O_i}$$
- $\delta_{H1} = -0.0706 * (0.999 * (1 - 0.999)) = -0.0000705$
 - Similarly for H2: $(0.1*0.0469) + (1.17*-0.0394) = -0.0414$
 - $\delta_{H2} = -0.0414 * (0.067 * (1 - 0.067)) = -0.00259$

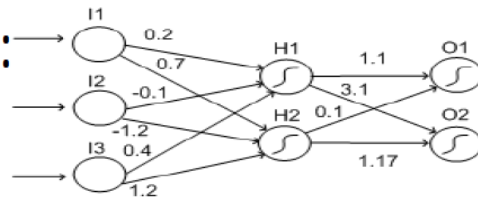
06/01/2023

EC-350 AI and DSS

EME (NUST)

34

A Worked Example:



Input unit	Hidden unit	η	δ_H	x_i	$\Delta = \eta * \delta_H * x_i$	Old weight	New weight
I1	H1	0.1	-0.0000705	10	-0.0000705	0.2	0.1999295
I1	H2	0.1	-0.00259	10	-0.00259	0.7	0.69741
I2	H1	0.1	-0.0000705	30	-0.0002115	-0.1	-0.1002115
I2	H2	0.1	-0.00259	30	-0.00777	-1.2	-1.20777
I3	H1	0.1	-0.0000705	20	-0.000141	0.4	0.39999
I3	H2	0.1	-0.00259	20	-0.00518	1.2	1.1948

Hidden unit	Output unit	η	δ_O	$h_i(E)$	$\Delta = \eta * \delta_O * h_i(E)$	Old weight	New weight
H1	O1	0.1	0.0469	0.999	0.000469	1.1	1.100469
H1	O2	0.1	-0.0394	0.999	-0.00394	3.1	3.0961
H2	O1	0.1	0.0469	0.0067	0.00314	0.1	0.10314
H2	O2	0.1	-0.0394	0.0067	-0.000264	1.17	1.16998

06/01/2023

EC-350 AI and DSS

EME (NUST)